UNIVERSIDAD DE COSTA RICA
Sede del Atlántico

RP Recinto de Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Laboratorio 16: Hardening Defensivo y Mitigación de Vulnerabilidades Web OWASP en Spring Boot 3 con Spring Security 6 ("MediVault Web")

Resumen

En este laboratorio guiado, usted asumirá el rol de Ingeniero(a) de Seguridad Back-End para la plataforma hospitalaria **MediVault Web**. Su misión técnica será transformar una API RESTful vulnerable en una solución empresarial altamente resistente frente a las amenazas más críticas del estándar **OWASP Top 10** y **OWASP API Security Top 10**.

A lo largo de la práctica, usted diseñará e implementará una arquitectura defensiva profunda en **Java 25** con **Spring Boot 3.3+** y **Spring Security 6**, incorporando:

1. Mitigación contra **Broken Access Control (IDOR / BOLA)** mediante validación estricta de propiedad de recursos (*Resource Ownership*).
2. Sanitización y validación exhaustiva de contratos DTO con **Jakarta Bean Validation** y manejo estandarizado de excepciones bajo **RFC 7807** (`ProblemDetail`), eliminando fugas de información sensible (*Information Disclosure*).

3. Hardening de cabeceras de respuesta HTTP defensivas (**CSP Nivel 3**, **HSTS**, **X-Content-Type-Options: nosniff**, **X-Frame-Options: DENY** y políticas restrictivas de caché para datos médicos).
4. Protección contra ataques de denegación de servicio y fuerza bruta mediante un filtro de **Rate Limiting** basado en el algoritmo *Token Bucket* con **Bucket4j**.

Metadatos del Laboratorio

- **Tiempo estimado de desarrollo:** 3.5 horas (210 minutos).
- **Herramientas requeridas:** Java OpenJDK 25 (o 21 LTS), Spring Boot 3.3+, Apache Maven 3.9+, H2 Database (en memoria), cliente cURL / Postman / Thunder Client, IDE (Visual Studio Code o IntelliJ IDEA), Git.
- **Metas de Aprendizaje:**

  1. Configurar una cadena de seguridad personalizada en Spring Security 6 (`SecurityFilterChain`) que inyecte cabeceras defensivas HTTP estándar y gestione sesiones estrictamente sin estado (*stateless*) mediante tokens JWT.

  2. Identificar, reproducir y mitigar vulnerabilidades de control de acceso a nivel de objeto (IDOR / BOLA) en capas de servicio y controladores, garantizando que ningún usuario acceda a registros confidenciales ajenos.

  3. Implementar defensas activas de entrada y tasa de transferencia: validaciones declarativas con Jakarta Bean Validation, captura centralizada de excepciones seguras y control de peticiones concurrentes con Bucket4j.

Contexto de Negocio: MediVault Web

La red de clínicas y hospitales **MediVault** almacena expedientes médicos digitales que contienen información de máxima confidencialidad: diagnósticos crónicos, tratamientos farmacológicos y notas reservadas.

Una auditoría de ciberseguridad reciente detectó las siguientes fallas críticas en el backend previo:

- **Vulnerabilidad IDOR (Insecure Direct Object References):** Pacientes autenticados podían consultar expedientes de terceros simplemente cambiando el identificador numérico en la URL (`/api/v1/expedientes/{id}`).
- **Fuga de Información en Stack Traces:** Al enviar datos no conformes o IDs inexistentes, el servidor respondía con errores internos `500` exponiendo clases internas de Hibernate, consultas SQL y versiones del framework.
- **Ataques de Fuerza Bruta en Autenticación:** El endpoint `/api/v1/auth/login` no contaba con mecanismos de aceleración ni bloqueo temporal, permitiendo diccionarios automatizados de contraseñas.

- **Ausencia de Cabeceras Defensivas:** Los navegadores web no recibían directivas para prevenir la interpretación errónea de tipos MIME (*MIME Sniffing*), incrustación en `iframe` (*Clickjacking*) ni políticas de protección de scripts.

Usted deberá refactorizar y blindar el núcleo del sistema para garantizar el cumplimiento de estándares de seguridad en aplicaciones de salud.

Estructura General de Carpetas del Proyecto

Usted construirá y organizará la solución bajo la siguiente estructura modular:

```
medivault-security-backend/
|-- pom.xml
\-- src/
    \-- main/
        |-- java/ucr/ac/cr/medivault/
        |   |-- domain/
        |   |   |-- Usuario.java
        |   |   |-- Rol.java
        |   |   \-- ExpedienteMedico.java
        |   |-- data/
        |   |   |-- UsuarioRepository.java
        |   |   \-- ExpedienteMedicoRepository.java
        |   |-- business/
        |   |   |-- AuthService.java
        |   |   \-- ExpedienteService.java
        |   |-- controller/
        |   |   |-- AuthController.java
        |   |   |-- ExpedienteController.java
        |   |   \-- dto/
        |   |       |-- LoginRequestDto.java
        |   |       |-- LoginResponseDto.java
        |   |       |-- ExpedienteRequestDto.java
        |   |       \-- ExpedienteResponseDto.java
        |   |-- security/
        |   |   |-- SecurityConfig.java
        |   |   |-- JwtTokenProvider.java
        |   |   |-- JwtAuthenticationFilter.java
        |   |   \-- RateLimitingFilter.java
        |   |-- exception/
        |   |   |-- AccessDeniedException.java
        |   |   |-- ResourceNotFoundException.java
        |   |   \-- GlobalExceptionHandler.java
        |   \-- MedivaultApplication.java
        \-- resources/
            |-- application.yml
            |-- schema.sql
            \-- data.sql
```

Parte 1: Práctica Guiada (Paso a Paso)

Paso 1: Configuración de Dependencias Maven (`pom.xml`)

Cree el archivo `pom.xml` con las dependencias necesarias para Spring Security, Jakarta Validation, soporte JWT (JJWT) y control de tasa (Bucket4j).

```
<?xml version="1.0" encoding="UTF-8"?>
<project
  xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi=
    "http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>
      spring-boot-starter-parent
    </artifactId>
    <version>3.3.4</version>
    <relativePath/>
  </parent>

  <groupId>ucr.ac.cr</groupId>
  <artifactId>medivault-security-backend</artifactId>
  <version>1.0.0</version>
  <name>medivault-security-backend</name>
  <description>API Segura MediVault</description>

  <properties>
    <java.version>25</java.version>
    <jjwt.version>0.12.5</jjwt.version>
    <bucket4j.version>8.10.1</bucket4j.version>
  </properties>

  <dependencies>
    <!-- Web y Seguridad -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>
        spring-boot-starter-web
      </artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>
        spring-boot-starter-security
      </artifactId>
    </dependency>

    <!-- JPA y Validacion -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>
        spring-boot-starter-data-jpa
```

```
      </artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>
        spring-boot-starter-validation
      </artifactId>
    </dependency>

    <!-- Base de datos H2 en memoria -->
    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <scope>runtime</scope>
    </dependency>

    <!-- Librerias JJWT -->
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-api</artifactId>
      <version>${jjwt.version}</version>
    </dependency>
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-impl</artifactId>
      <version>${jjwt.version}</version>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-jackson</artifactId>
      <version>${jjwt.version}</version>
      <scope>runtime</scope>
    </dependency>

    <!-- Rate Limiting con Bucket4j -->
    <dependency>
      <groupId>com.bucket4j</groupId>
      <artifactId>bucket4j-core</artifactId>
      <version>${bucket4j.version}</version>
    </dependency>

    <!-- Testing -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>
        spring-boot-starter-test
      </artifactId>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework.security</groupId>
      <artifactId>spring-security-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>
```

```
          spring-boot-maven-plugin
        </artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

Paso 2: Configuración de la Aplicación (`application.yml`)

Configure la base de datos H2 en memoria y desactive la exposición de banners o información detallada del servidor en `src/main/resources/application.yml`:

```
server:
  port: 8080
  error:
    include-message: never
    include-binding-errors: never
    include-stacktrace: never
    include-exception: false

spring:
  application:
    name: medivault-security-backend
  datasource:
    url: >-
      jdbc:h2:mem:medivaultdb;
      DB_CLOSE_DELAY=-1;
      DB_CLOSE_ON_EXIT=FALSE
    driver-class-name: org.h2.Driver
    username: sa
    password: ""
  jpa:
    database-platform: >-
      org.hibernate.dialect.H2Dialect
    hibernate:
      ddl-auto: none
    show-sql: false
  sql:
    init:
      mode: always
      schema-locations: classpath:schema.sql
      data-locations: classpath:data.sql

# Parametros de seguridad JWT
medivault:
  security:
    jwt:
      # Clave secreta HMAC-SHA256 (32+ chars)
      secret: >-
        v9y$B&E)H@McQfTjWnZr4u7x!A%D*G-
        KaPdSgUkXp2s5v8y/B?E(H+MbQeThWmYq
      expiration-ms: 3600000 # 1 hora
```

> [!NOTE] Copie los archivos `schema.sql` y `data.sql` proporcionados en la subcarpeta `resources/sql/` directamente en la raíz de su carpeta `src/main/resources/`.

Paso 3: Entidades de Dominio y Repositorios JPA

Enumeración de Roles (`Rol.java`)

Cree `src/main/java/ucr/ac/cr/medivault/domain/Rol.java`:

```
package ucr.ac.cr.medivault.domain;

public enum Rol {
    ROLE_ADMIN,
    ROLE_DOCTOR,
    ROLE_PACIENTE
}
```

Entidad Usuario (`Usuario.java`)

Cree `src/main/java/ucr/ac/cr/medivault/domain/Usuario.java`:

```
package ucr.ac.cr.medivault.domain;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "USUARIOS")
public class Usuario {

    @Id
    @GeneratedValue(
        strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false,
            unique = true,
            length = 50)
    private String username;

    @Column(name = "password_hash",
            nullable = false,
            length = 100)
    private String passwordHash;

    @Column(name = "nombre_completo",
            nullable = false,
            length = 100)
    private String nombreCompleto;

    @Column(nullable = false,
            unique = true,
            length = 100)
    private String email;
```

```
    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private Rol rol;

    @Column(nullable = false, length = 20)
    private String estado = "ACTIVO";

    @Column(name = "fecha_creacion",
            insertable = false,
            updatable = false)
    private LocalDateTime fechaCreacion;

    public Usuario() {}

    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }

    public String getUsername() {
        return username;
    }
    public void setUsername(String username) {
        this.username = username;
    }

    public String getPasswordHash() {
        return passwordHash;
    }
    public void setPasswordHash(String hash) {
        this.passwordHash = hash;
    }

    public String getNombreCompleto() {
        return nombreCompleto;
    }
    public void setNombreCompleto(String n) {
        this.nombreCompleto = n;
    }

    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }

    public Rol getRol() {
        return rol;
    }
    public void setRol(Rol rol) {
        this.rol = rol;
    }

    public String getEstado() {
        return estado;
    }
    public void setEstado(String estado) {
        this.estado = estado;
    }
```

```
}
```

Entidad Expediente Médico (`ExpedienteMedico.java`)

Cree `src/main/java/ucr/ac/cr/medivault/domain/ExpedienteMedico.java`:

```
package ucr.ac.cr.medivault.domain;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "EXPEDIENTES_MEDICOS")
public class ExpedienteMedico {

    @Id
    @GeneratedValue(
        strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY,
               optional = false)
    @JoinColumn(name = "paciente_id",
                nullable = false)
    private Usuario paciente;

    @ManyToOne(fetch = FetchType.LAZY,
               optional = false)
    @JoinColumn(name = "medico_asignado_id",
                nullable = false)
    private Usuario medicoAsignado;

    @Column(name = "codigo_expediente",
            nullable = false,
            unique = true,
            length = 30)
    private String codigoExpediente;

    @Column(name = "diagnostico_principal",
            nullable = false)
    private String diagnosticoPrincipal;

    @Column(name = "notas_clinicas",
            nullable = false,
            columnDefinition = "TEXT")
    private String notasClinicas;

    @Column(name = "nivel_confidencialidad",
            nullable = false,
            length = 20)
    private String nivelConfidencialidad;

    @Column(name = "fecha_registro",
            insertable = false,
            updatable = false)
    private LocalDateTime fechaRegistro;

    public ExpedienteMedico() {}
```

```
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }

    public Usuario getPaciente() {
        return paciente;
    }
    public void setPaciente(Usuario paciente) {
        this.paciente = paciente;
    }

    public Usuario getMedicoAsignado() {
        return medicoAsignado;
    }
    public void setMedicoAsignado(Usuario m) {
        this.medicoAsignado = m;
    }

    public String getCodigoExpediente() {
        return codigoExpediente;
    }
    public void setCodigoExpediente(String c) {
        this.codigoExpediente = c;
    }

    public String getDiagnosticoPrincipal() {
        return diagnosticoPrincipal;
    }
    public void setDiagnosticoPrincipal(String d) {
        this.diagnosticoPrincipal = d;
    }

    public String getNotasClinicas() {
        return notasClinicas;
    }
    public void setNotasClinicas(String notas) {
        this.notasClinicas = notas;
    }

    public String getNivelConfidencialidad() {
        return nivelConfidencialidad;
    }
    public void setNivelConfidencialidad(String n) {
        this.nivelConfidencialidad = n;
    }

    public LocalDateTime getFechaRegistro() {
        return fechaRegistro;
    }
}
```

Repositorios JPA

Cree `src/main/java/ucr/ac/cr/medivault/data/UsuarioRepository.java`:

```
package ucr.ac.cr.medivault.data;

import org.springframework.data.jpa.repository.*;
import org.springframework.stereotype.Repository;
import ucr.ac.cr.medivault.domain.Usuario;
import java.util.Optional;

@Repository
public interface UsuarioRepository
        extends JpaRepository<Usuario, Long> {

    Optional<Usuario> findByUsername(
            String username);

    boolean existsByUsername(String username);
}
```

Cree `src/main/java/ucr/ac/cr/medivault/data/ExpedienteMedicoRepository.java`:

```
package ucr.ac.cr.medivault.data;

import org.springframework.data.jpa.repository.*;
import org.springframework.data.repository.query.*;
import org.springframework.stereotype.Repository;
import ucr.ac.cr.medivault.domain.ExpedienteMedico;
import java.util.List;
import java.util.Optional;

@Repository
public interface ExpedienteMedicoRepository
        extends JpaRepository<ExpedienteMedico, Long> {

    List<ExpedienteMedico> findByPacienteId(
            Long pacienteId);

    // Consulta JPA parametrizada contra SQLi
    @Query("SELECT e FROM ExpedienteMedico e " +
           "JOIN FETCH e.paciente " +
           "JOIN FETCH e.medicoAsignado " +
           "WHERE e.id = :id")
    Optional<ExpedienteMedico> findByIdWithDetails(
            @Param("id") Long id);
}
```

Paso 4: Capa de Seguridad JWT

Proveedor Criptográfico de Tokens (`JwtTokenProvider.java`)

Cree `src/main/java/ucr/ac/cr/medivault/security/JwtTokenProvider.java`:

```
package ucr.ac.cr.medivault.security;

import io.jsonwebtoken.*;
```

```
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation
        .Value;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;

@Component
public class JwtTokenProvider {

    private final SecretKey key;
    private final long expirationMs;

    public JwtTokenProvider(
            @Value("${medivault.security.jwt.secret}")
            String secret,
            @Value("${medivault.security" +
       ".jwt.expiration-ms}")
            long expirationMs) {
        byte[] bytes = secret.getBytes(
                StandardCharsets.UTF_8);
        this.key = Keys.hmacShaKeyFor(bytes);
        this.expirationMs = expirationMs;
    }

    public String generateToken(String username,
                                String role) {
        Date now = new Date();
        Date expiry = new Date(
                now.getTime() + expirationMs);

        return Jwts.builder()
                .subject(username)
                .claim("role", role)
                .issuedAt(now)
                .expiration(expiry)
                .signWith(key, Jwts.SIG.HS256)
                .compact();
    }

    public String getUsernameFromToken(
            String token) {
        Claims claims = Jwts.parser()
                .verifyWith(key)
                .build()
                .parseSignedClaims(token)
                .getPayload();
        return claims.getSubject();
    }

    public boolean validateToken(String token) {
        try {
            Jwts.parser()
                .verifyWith(key)
                .build()
                .parseSignedClaims(token);
            return true;
        } catch (JwtException |
                 IllegalArgumentException e) {
            return false;
        }
```

```
    }
}
```

Filtro de Autenticación JWT (`JwtAuthenticationFilter.java`)

Cree `src/main/java/ucr/ac/cr/medivault/security/JwtAuthenticationFilter.java`:

```
package ucr.ac.cr.medivault.security;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import org.springframework.security.authentication.*;
import org.springframework.security.core.authority.*;
import org.springframework.security.core.context.*;
import org.springframework.stereotype.Component;
import org.springframework.util.StringUtils;
import org.springframework.web.filter
        .OncePerRequestFilter;
import ucr.ac.cr.medivault.data.UsuarioRepository;
import ucr.ac.cr.medivault.domain.Usuario;

import java.io.IOException;
import java.util.Collections;

@Component
public class JwtAuthenticationFilter
        extends OncePerRequestFilter {

    private final JwtTokenProvider tokenProvider;
    private final UsuarioRepository userRepo;

    public JwtAuthenticationFilter(
            JwtTokenProvider tokenProvider,
            UsuarioRepository userRepo) {
        this.tokenProvider = tokenProvider;
        this.userRepo = userRepo;
    }

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain)
            throws ServletException, IOException {

        String jwt = resolveToken(request);

        if (StringUtils.hasText(jwt) &&
            tokenProvider.validateToken(jwt)) {
            String username = tokenProvider
                    .getUsernameFromToken(jwt);

            Usuario u = userRepo
                    .findByUsername(username)
                    .orElse(null);

            if (u != null &&
                "ACTIVO".equals(u.getEstado())) {
                var role =
```

```
                    new SimpleGrantedAuthority(
                        u.getRol().name());

                var auth =
                    new
                    UsernamePasswordAuthenticationToken(
                        u.getUsername(), null,
                        Collections.singletonList(role));

                SecurityContextHolder.getContext()
                        .setAuthentication(auth);
            }
        }

        filterChain.doFilter(request, response);
    }

    private String resolveToken(
            HttpServletRequest req) {
        String bearer = req.getHeader(
                "Authorization");
        if (StringUtils.hasText(bearer) &&
            bearer.startsWith("Bearer ")) {
            return bearer.substring(7);
        }
        return null;
    }
}
```

Paso 5: Configuración Central de Seguridad y Cabeceras Defensivas (`SecurityConfig.java`)

Configure en Spring Security 6 la gestión sin estado (*stateless*), desactive CSRF (por tratarse de una API desacoplada que usa Bearer tokens), restrinja los endpoints públicos y establezca un conjunto estricto de **cabeceras de seguridad HTTP**:

Cree `src/main/java/ucr/ac/cr/medivault/security/SecurityConfig.java`:

```
package ucr.ac.cr.medivault.security;

import org.springframework.context.annotation.*;
import org.springframework.http.HttpMethod;
import org.springframework.security.config
        .annotation.method.configuration
        .EnableMethodSecurity;
import org.springframework.security.config
        .annotation.web.builders.HttpSecurity;
import org.springframework.security.config
        .annotation.web.configuration
        .EnableWebSecurity;
import org.springframework.security.config
        .annotation.web.configurers.*;
import org.springframework.security.config
        .http.SessionCreationPolicy;
```

```
import org.springframework.security.crypto.bcrypt
        .BCryptPasswordEncoder;
import org.springframework.security.crypto.password
        .PasswordEncoder;
import org.springframework.security.web
        .SecurityFilterChain;
import org.springframework.security.web
        .authentication
        .UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtFilter;
    private final RateLimitingFilter rateFilter;

    public SecurityConfig(
            JwtAuthenticationFilter jwtFilter,
            RateLimitingFilter rateFilter) {
        this.jwtFilter = jwtFilter;
        this.rateFilter = rateFilter;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(10);
    }

    @Bean
    public SecurityFilterChain filterChain(
            HttpSecurity http) throws Exception {
        http
            // 1. Desactivacion CSRF (Stateless JWT)
            .csrf(AbstractHttpConfigurer::disable)

            // 2. Sesion sin estado (Stateless)
            .sessionManagement(s -> s
                .sessionCreationPolicy(
                    SessionCreationPolicy.STATELESS
                )
            )

            // 3. Reglas de Autorizacion
            .authorizeHttpRequests(a -> a
                .requestMatchers(
                    HttpMethod.POST,
                    "/api/v1/auth/login"
                ).permitAll()
                .requestMatchers("/h2-console/**")
                .denyAll()
                .requestMatchers(
                    "/api/v1/expedientes/**")
                .authenticated()
                .anyRequest().authenticated()
            )

            // 4. Hardening de Cabeceras HTTP
            .headers(h -> h
                // Bloquea Clickjacking (no iframes)
                .frameOptions(
                    HeadersConfigurer
```

```
                        .FrameOptionsConfig::deny
                )
                // Bloquea MIME-Sniffing
                .contentTypeOptions(
                    HeadersConfigurer
                        .ContentTypeOptionsConfig
                        ::disable
                )
                // CSP Nivel 3
                .contentSecurityPolicy(csp -> csp
                    .policyDirectives(
                        "default-src 'self'; " +
                        "frame-ancestors 'none'; " +
                        "object-src 'none';"
                    )
                )
                // HSTS: obliga uso de HTTPS
                .httpStrictTransportSecurity(hsts ->
                    hsts.includeSubDomains(true)
                        .maxAgeInSeconds(31536000)
                )
                // Prohibe cache de datos medicos
                .cacheControl(
                    HeadersConfigurer
                        .CacheControlConfig::disable
                )
            )

            // 5. Insercion de Filtros en Pipeline
            .addFilterBefore(
                rateFilter,
                UsernamePasswordAuthenticationFilter
                    .class
            )
            .addFilterBefore(
                jwtFilter,
                UsernamePasswordAuthenticationFilter
                    .class
            );

        return http.build();
    }
}
```

Paso 6: Filtro de Rate Limiting con Bucket4j (`RateLimitingFilter.java`)

Para mitigar ataques de fuerza bruta hacia la autenticación y saturación de servicios, implemente un filtro que reserve un cubo (*bucket*) con una tasa máxima de 10 peticiones por minuto para login y 30 para consultas generales:

Cree `src/main/java/ucr/ac/cr/medivault/security/RateLimitingFilter.java`:

```
package ucr.ac.cr.medivault.security;

import io.github.bucket4j.*;
```

```
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.stereotype.Component;
import org.springframework.web.filter
        .OncePerRequestFilter;

import java.io.IOException;
import java.time.Duration;

@Component
public class RateLimitingFilter
        extends OncePerRequestFilter {

    // Login: 10 peticiones por minuto
    private final Bucket authBucket = Bucket.builder()
            .addLimit(Bandwidth.classic(10,
                Refill.intervally(10,
                    Duration.ofMinutes(1))))
            .build();

    // Expedientes: 30 peticiones por minuto
    private final Bucket apiBucket = Bucket.builder()
            .addLimit(Bandwidth.classic(30,
                Refill.intervally(30,
                    Duration.ofMinutes(1))))
            .build();

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain)
            throws ServletException, IOException {

        String path = request.getRequestURI();
        Bucket target = null;

        if (path.startsWith("/api/v1/auth/login")) {
            target = authBucket;
        } else if (path.startsWith(
                "/api/v1/expedientes")) {
            target = apiBucket;
        }

        if (target != null) {
            if (target.tryConsume(1)) {
                filterChain.doFilter(
                    request, response);
            } else {
                // Rate Limit superado: HTTP 429
                response.setStatus(
                    HttpStatus
                        .TOO_MANY_REQUESTS.value()
                );
                response.setContentType(
                    MediaType
                        .APPLICATION_PROBLEM_JSON_VALUE
                );
                response.setHeader(
                    "Retry-After", "60");

```

```
                String json = """
                    {
                      "type": "%s",
                      "title": "%s",
                      "status": 429,
                      "detail": "%s",
                      "instance": "%s"
                    }
                    """.formatted(
                        "https://medivault.cr/rl",
                        "Demasiadas Peticiones",
                        "Limite superado. Espere 60s.",
                        path
                    );

                response.getWriter().write(json);
            }
        } else {
            filterChain.doFilter(request, response);
        }
    }
}
```

Paso 7: DTOs con Validación Declarativa y Manejo Centralizado RFC 7807

DTOs de Autenticación

Cree `src/main/java/ucr/ac/cr/medivault/controller/dto/LoginRequestDto.java`:

```
package ucr.ac.cr.medivault.controller.dto;

import jakarta.validation.constraints.*;

public record LoginRequestDto(
    @NotBlank(message = "Usuario obligatorio.")
    @Size(min = 4, max = 50,
          message = "Usuario: 4 a 50 caracteres.")
    @Pattern(regexp = "^[a-zA-Z0-9_]+$",
             message = "Solo alfanumericos.")
    String username,

    @NotBlank(message = "Contrasena requerida.")
    @Size(min = 8, max = 100,
          message = "Contrasena minimo 8 caracteres.")
    String password
) {}
```

Cree `src/main/java/ucr/ac/cr/medivault/controller/dto/LoginResponseDto.java`:

```
package ucr.ac.cr.medivault.controller.dto;

public record LoginResponseDto(
    String token,
```

```
    String tokenType,
    long expiresInMs,
    String username,
    String rol
) {}
```

DTOs de Expedientes Médicos

Cree `src/main/java/ucr/ac/cr/medivault/controller/dto/ExpedienteRequestDto.java`:

```
package ucr.ac.cr.medivault.controller.dto;

import jakarta.validation.constraints.*;

public record ExpedienteRequestDto(
    @NotNull(message = "ID paciente obligatorio.")
    @Positive(
        message = "ID paciente positivo.")
    Long pacienteId,

    @NotBlank(message = "Codigo obligatorio.")
    @Pattern(regexp = "^EXP-\\d{4}-\\d{3,6}$",
             message = "Formato: EXP-YYYY-NNN.")
    String codigoExpediente,

    @NotBlank(message = "Diagnostico obligatorio.")
    @Size(min = 5, max = 255,
          message = "Diagnostico: 5 a 255 caracteres.")
    String diagnosticoPrincipal,

    @NotBlank(message = "Notas clinicas requeridas.")
    @Size(min = 10, max = 4000,
          message = "Notas: 10 a 4000 caracteres.")
    String notasClinicas,

    @NotBlank(message = "Confidencialidad requerida.")
    @Pattern(
        regexp = "^(BAJO|MEDIO|ALTO)$",
        message = "Valores: BAJO, MEDIO, ALTO.")
    String nivelConfidencialidad
) {}
```

Cree `src/main/java/ucr/ac/cr/medivault/controller/dto/ExpedienteResponseDto.java`:

```
package ucr.ac.cr.medivault.controller.dto;

import java.time.LocalDateTime;

public record ExpedienteResponseDto(
    Long id,
    String codigoExpediente,
    String pacienteNombre,
    String medicoNombre,
    String diagnosticoPrincipal,
    String notasClinicas,
    String nivelConfidencialidad,
    LocalDateTime fechaRegistro
```

```
) {}
```

Excepciones Personalizadas de Seguridad

Cree `src/main/java/ucr/ac/cr/medivault/exception/AccessDeniedSecurityException.java`:

```
package ucr.ac.cr.medivault.exception;

public class AccessDeniedSecurityException
        extends RuntimeException {
    public AccessDeniedSecurityException(String msg) {
        super(msg);
    }
}
```

Cree `src/main/java/ucr/ac/cr/medivault/exception/ResourceNotFoundException.java`:

```
package ucr.ac.cr.medivault.exception;

public class ResourceNotFoundException
        extends RuntimeException {
    public ResourceNotFoundException(String msg) {
        super(msg);
    }
}
```

Manejador Global RFC 7807 (`GlobalSecurityExceptionHandler.java`)

Para evitar la fuga de stack traces o nombres de tablas, capture centralizadamente las fallas retornando objetos estandarizados `ProblemDetail`:

Cree `src/main/java/ucr/ac/cr/medivault/exception/GlobalSecurityExceptionHandler.java`:

```
package ucr.ac.cr.medivault.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.security.authentication
        .BadCredentialsException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind
        .MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;

import java.net.URI;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalSecurityExceptionHandler {

    @ExceptionHandler(
```

```
        AccessDeniedSecurityException.class)
    public ProblemDetail handleAccessDenied(
            AccessDeniedSecurityException ex) {
        ProblemDetail p = ProblemDetail
                .forStatusAndDetail(
                    HttpStatus.FORBIDDEN,
                    ex.getMessage()
                );
        p.setTitle("Acceso Prohibido (IDOR / BOLA)");
        p.setType(URI.create(
            "https://medivault.ucr.ac.cr/forbidden"
        ));
        return p;
    }

    @ExceptionHandler(ResourceNotFoundException.class)
    public ProblemDetail handleNotFound(
            ResourceNotFoundException ex) {
        ProblemDetail p = ProblemDetail
                .forStatusAndDetail(
                    HttpStatus.NOT_FOUND,
                    ex.getMessage()
                );
        p.setTitle("Recurso No Encontrado");
        p.setType(URI.create(
            "https://medivault.ucr.ac.cr/not-found"
        ));
        return p;
    }

    @ExceptionHandler(BadCredentialsException.class)
    public ProblemDetail handleBadCredentials(
            BadCredentialsException ex) {
        ProblemDetail p = ProblemDetail
                .forStatusAndDetail(
                    HttpStatus.UNAUTHORIZED,
                    "Credenciales invalidas."
                );
        p.setTitle("Fallo de Autenticacion");
        p.setType(URI.create(
            "https://medivault.ucr.ac.cr/unauthorized"
        ));
        return p;
    }

    @ExceptionHandler(
        MethodArgumentNotValidException.class)
    public ProblemDetail handleValidation(
            MethodArgumentNotValidException ex) {
        ProblemDetail p = ProblemDetail
                .forStatusAndDetail(
                    HttpStatus.BAD_REQUEST,
                    "Validacion de datos fallida."
                );
        p.setTitle("Error de Validacion");
        p.setType(URI.create(
            "https://medivault.ucr.ac.cr/bad-request"
        ));

        Map<String, String> errors = new HashMap<>();
        for (FieldError fe :
                ex.getBindingResult()
                    .getFieldErrors()) {
```

```
            errors.put(fe.getField(),
                       fe.getDefaultMessage());
        }
        p.setProperty("invalid_fields", errors);
        return p;
    }

    @ExceptionHandler(Exception.class)
    public ProblemDetail handleGeneric(Exception ex) {
        ProblemDetail p = ProblemDetail
                .forStatusAndDetail(
                    HttpStatus.INTERNAL_SERVER_ERROR,
                    "Error interno del servidor."
                );
        p.setTitle("Error Interno");
        p.setType(URI.create(
            "https://medivault.ucr.ac.cr/internal"
        ));
        return p;
    }
}
```

Paso 8: Capa de Negocio con Mitigación Activa de IDOR / BOLA

En el servicio de expedientes, aplique la regla de **Ownership Verification**: * Un usuario con rol `ROLE_PACIENTE` **únicamente** puede acceder a los expedientes cuyo `paciente.username` coincida exactamente con el usuario autenticado. * Un usuario con rol `ROLE_DOCTOR` puede acceder si es el médico asignado al expediente. * Los administradores (`ROLE_ADMIN`) tienen acceso para auditoría. * Cualquier intento que infrinja estas reglas lanza inmediatamente una `AccessDeniedSecurityException`.

Cree `src/main/java/ucr/ac/cr/medivault/business/ExpedienteService.java`:

```
package ucr.ac.cr.medivault.business;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation
        .Transactional;
import ucr.ac.cr.medivault.controller.dto.*;
import ucr.ac.cr.medivault.data.*;
import ucr.ac.cr.medivault.domain.*;
import ucr.ac.cr.medivault.exception.*;

import java.util.List;

@Service
public class ExpedienteService {

    private final ExpedienteMedicoRepository expRepo;
    private final UsuarioRepository userRepo;

    public ExpedienteService(
            ExpedienteMedicoRepository expRepo,
            UsuarioRepository userRepo) {
        this.expRepo = expRepo;
```

```
        this.userRepo = userRepo;
    }

    @Transactional(readOnly = true)
    public ExpedienteResponseDto consultarPorId(
            Long expedienteId,
            String currentUsername) {
        ExpedienteMedico exp = expRepo
                .findByIdWithDetails(expedienteId)
                .orElseThrow(() ->
                    new ResourceNotFoundException(
                        "Expediente no existe: " +
                        expedienteId));

        Usuario solicitante = userRepo
                .findByUsername(currentUsername)
                .orElseThrow(() ->
                    new AccessDeniedSecurityException(
                        "Usuario no reconocido."));

        // Verificacion de Propiedad (IDOR / BOLA)
        validarPropiedadRecurso(exp, solicitante);

        return mapToResponseDto(exp);
    }

    @Transactional(readOnly = true)
    public List<ExpedienteResponseDto>
            listarExpedientesPropios(
                String currentUsername) {
        Usuario solicitante = userRepo
                .findByUsername(currentUsername)
                .orElseThrow(() ->
                    new AccessDeniedSecurityException(
                        "Usuario no reconocido."));

        return expRepo
                .findByPacienteId(solicitante.getId())
                .stream()
                .map(this::mapToResponseDto)
                .toList();
    }

    @Transactional
    public ExpedienteResponseDto registrarExpediente(
            ExpedienteRequestDto dto,
            String doctorUsername) {
        Usuario doc = userRepo
                .findByUsername(doctorUsername)
                .orElseThrow(() ->
                    new AccessDeniedSecurityException(
                        "Doctor no registrado."));

        if (doc.getRol() != Rol.ROLE_DOCTOR &&
            doc.getRol() != Rol.ROLE_ADMIN) {
            throw new AccessDeniedSecurityException(
                "Solo medicos autorizados " +
                "pueden registrar expedientes.");
        }

        Usuario pac = userRepo
                .findById(dto.pacienteId())
                .orElseThrow(() ->
```

```
                    new ResourceNotFoundException(
                        "Paciente no existe: " +
                        dto.pacienteId()));

        ExpedienteMedico e = new ExpedienteMedico();
        e.setPaciente(pac);
        e.setMedicoAsignado(doc);
        e.setCodigoExpediente(dto.codigoExpediente());
        e.setDiagnosticoPrincipal(
            dto.diagnosticoPrincipal());
        e.setNotasClinicas(dto.notasClinicas());
        e.setNivelConfidencialidad(
            dto.nivelConfidencialidad());

        ExpedienteMedico guardado = expRepo.save(e);
        return mapToResponseDto(guardado);
    }

    /**
     * Regla Defensiva de Control de Acceso
     */
    private void validarPropiedadRecurso(
            ExpedienteMedico exp,
            Usuario solicitante) {
        // Regla A: Administrador auditor
        if (solicitante.getRol() == Rol.ROLE_ADMIN) {
            return;
        }

        // Regla B: Paciente dueno del expediente
        if (solicitante.getRol() ==
                Rol.ROLE_PACIENTE) {
            if (!exp.getPaciente().getId()
                    .equals(solicitante.getId())) {
                throw new
                    AccessDeniedSecurityException(
                        "Violacion IDOR: Sin acceso.");
            }
            return;
        }

        // Regla C: Medico asignado al expediente
        if (solicitante.getRol() == Rol.ROLE_DOCTOR) {
            if (!exp.getMedicoAsignado().getId()
                    .equals(solicitante.getId())) {
                throw new
                    AccessDeniedSecurityException(
                        "Violacion BOLA: No asignado.");
            }
        }
    }

    private ExpedienteResponseDto mapToResponseDto(
            ExpedienteMedico e) {
        return new ExpedienteResponseDto(
                e.getId(),
                e.getCodigoExpediente(),
                e.getPaciente().getNombreCompleto(),
                e.getMedicoAsignado()
                        .getNombreCompleto(),
                e.getDiagnosticoPrincipal(),
                e.getNotasClinicas(),
                e.getNivelConfidencialidad(),
```

```
                e.getFechaRegistro()
        );
    }
}
```

Cree también `src/main/java/ucr/ac/cr/medivault/business/AuthService.java`:

```
package ucr.ac.cr.medivault.business;

import org.springframework.security.authentication
        .BadCredentialsException;
import org.springframework.security.crypto.password
        .PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation
        .Transactional;
import ucr.ac.cr.medivault.controller.dto.*;
import ucr.ac.cr.medivault.data.UsuarioRepository;
import ucr.ac.cr.medivault.domain.Usuario;
import ucr.ac.cr.medivault.security.JwtTokenProvider;

@Service
public class AuthService {

    private final UsuarioRepository userRepo;
    private final PasswordEncoder encoder;
    private final JwtTokenProvider jwtProvider;

    public AuthService(UsuarioRepository userRepo,
                       PasswordEncoder encoder,
                       JwtTokenProvider jwtProvider) {
        this.userRepo = userRepo;
        this.encoder = encoder;
        this.jwtProvider = jwtProvider;
    }

    @Transactional(readOnly = true)
    public LoginResponseDto autenticar(
            LoginRequestDto request) {
        Usuario u = userRepo
                .findByUsername(request.username())
                .orElseThrow(() ->
                    new BadCredentialsException(
                        "Credenciales invalidas."));

        if (!encoder.matches(request.password(),
                             u.getPasswordHash())) {
            throw new BadCredentialsException(
                "Credenciales invalidas.");
        }

        if (!"ACTIVO".equals(u.getEstado())) {
            throw new BadCredentialsException(
                "Cuenta inactiva o bloqueada.");
        }

        String token = jwtProvider.generateToken(
                u.getUsername(),
                u.getRol().name());
```

```
        return new LoginResponseDto(
                token,
                "Bearer",
                3600000,
                u.getUsername(),
                u.getRol().name()
        );
    }
}
```

Paso 9: Controladores RESTful

Cree `src/main/java/ucr/ac/cr/medivault/controller/AuthController.java`:

```
package ucr.ac.cr.medivault.controller;

import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import ucr.ac.cr.medivault.business.AuthService;
import ucr.ac.cr.medivault.controller.dto.*;

@RestController
@RequestMapping("/api/v1/auth")
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/login")
    public ResponseEntity<LoginResponseDto> login(
            @Valid @RequestBody LoginRequestDto req) {
        return ResponseEntity.ok(
            authService.autenticar(req)
        );
    }
}
```

Cree `src/main/java/ucr/ac/cr/medivault/controller/ExpedienteController.java`:

```
package ucr.ac.cr.medivault.controller;

import jakarta.validation.Valid;
import org.springframework.http.*;
import org.springframework.security.core.*;
import org.springframework.web.bind.annotation.*;
import ucr.ac.cr.medivault.business.ExpedienteService;
import ucr.ac.cr.medivault.controller.dto.*;

import java.util.List;
```

```
@RestController
@RequestMapping("/api/v1/expedientes")
public class ExpedienteController {

    private final ExpedienteService expService;

    public ExpedienteController(
            ExpedienteService expService) {
        this.expService = expService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<ExpedienteResponseDto>
            obtenerPorId(
                @PathVariable Long id,
                Authentication auth) {
        String user = auth.getName();
        return ResponseEntity.ok(
            expService.consultarPorId(id, user)
        );
    }

    @GetMapping("/mis-expedientes")
    public ResponseEntity<
            List<ExpedienteResponseDto>>
            obtenerMisExpedientes(
                Authentication auth) {
        String user = auth.getName();
        return ResponseEntity.ok(
            expService.listarExpedientesPropios(user)
        );
    }

    @PostMapping
    public ResponseEntity<ExpedienteResponseDto>
            crearExpediente(
                @Valid @RequestBody
                ExpedienteRequestDto req,
                Authentication auth) {
        String doc = auth.getName();
        ExpedienteResponseDto creado =
            expService.registrarExpediente(req, doc);
        return ResponseEntity.status(
            HttpStatus.CREATED
        ).body(creado);
    }
}
```

Cree la clase principal `src/main/java/ucr/ac/cr/medivault/MedivaultApplication.java`:

```
package ucr.ac.cr.medivault;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure
        .SpringBootApplication;

@SpringBootApplication
public class MedivaultApplication {
    public static void main(String[] args) {
        SpringApplication.run(
```

```
            MedivaultApplication.class,
            args
        );
    }
}
```

Paso 10: Verificación Práctica y Evidencia de Seguridad con cURL

Inicie la aplicación ejecutando en su terminal:

```
mvn spring-boot:run
```

Abra una segunda terminal y ejecute las siguientes cinco pruebas de seguridad:

Prueba 1: Autenticación Exitosa y Obtención de Tokens JWT

Inicie sesión con las cuentas de dos pacientes distintos:

```
# 1. Login con paciente_carlos
curl -X POST \
  http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username": "paciente_carlos",
       "password": "Password123!"}'

# 2. Login con paciente_maria
curl -X POST \
  http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username": "paciente_maria",
       "password": "Password123!"}'
```

Salida esperada: Un código `200 OK` con el token JWT firmado en formato compacto.

Prueba 2: Inspección de Cabeceras Defensivas HTTP

Verifique que el servidor inyecta las cabeceras de endurecimiento (*hardening*):

```
curl -I http://localhost:8080/api/v1/auth/login
```

Salida esperada:

```
HTTP/1.1 200 OK
Content-Security-Policy: default-src 'self'
Strict-Transport-Security: max-age=31536000
X-Frame-Options: DENY
```

```
```

Prueba 3: Consulta Legítima de Expediente Propio

Consulte el expediente `101` utilizando el token de `paciente_carlos` (dueño legítimo):

```
curl -X GET \
  http://localhost:8080/api/v1/expedientes/101 \
  -H "Authorization: Bearer <TOKEN_CARLOS>"
```

Salida esperada: Código `200 OK` con los datos clínicos del expediente 101.

Prueba 4: Intento de Ataque IDOR / BOLA Bloqueado

Utilice el token de `paciente_maria` para intentar acceder al expediente `101` (perteneciente a Carlos):

```
curl -X GET \
  http://localhost:8080/api/v1/expedientes/101 \
  -H "Authorization: Bearer <TOKEN_MARIA>"
```

Salida esperada: Código `403 Forbidden` estructurado bajo RFC 7807:

```
{
  "type": "https://medivault.ucr.ac.cr/forbidden",
  "title": "Acceso Prohibido (IDOR / BOLA)",
  "status": 403,
  "detail": "Violacion IDOR: Acceso no autorizado."
}
```

Prueba 5: Bloqueo de Ataque de Fuerza Bruta (Rate Limiting)

Ejecute más de 10 peticiones seguidas al endpoint de login para activar el filtro de Bucket4j:

```
for i in {1..12}; do
  curl -s -o /dev/null -w "%{http_code}\n" \
    -X POST \
    http://localhost:8080/api/v1/auth/login \
    -H "Content-Type: application/json" \
    -d '{"username": "admin_root",
         "password": "err"}'
done
```

Salida esperada: Las primeras 10 peticiones devolverán `401 Unauthorized`, pero a partir de la 11 el servidor devolverá inmediatamente `429 Too Many Requests` con la cabecera `Retry-After: 60`.

Parte 2: Depuración y Análisis Forense de Fallas de Seguridad

Planteamiento del Problema

Imagine que un desarrollador novato del equipo de MediVault implementó un nuevo método en el controlador para actualizar notas médicas rápidas:

```
 // CODIGO DEFECTUOSO INTENCIONAL (NO COPIAR)
@PatchMapping("/{id}/notas")
public ResponseEntity<Void>
        actualizarNotasVulnerable(
        @PathVariable Long id,
        @RequestBody String nuevasNotas) {
    // FALLA: Omite autenticacion y propiedad
    ExpedienteMedico exp = expRepo
            .findById(id).orElseThrow();
    exp.setNotasClinicas(nuevasNotas);
    expRepo.save(exp);
    return ResponseEntity.noContent().build();
}
```

Procedimiento de Análisis y Depuración

- [ ] **1. Identificar la brecha:** Observe que el método `@PatchMapping` no recibe el objeto `Authentication`, no valida si quien ejecuta la petición es el médico asignado al expediente, y además acepta cualquier cadena sin validación de longitud o sanitización.
- [ ] **2. Probar la vulnerabilidad:** Si un atacante ejecuta una petición `PATCH` al expediente de cualquier paciente con un cuerpo malicioso, el servidor sobreescribe la nota clínica sin dejar traza de auditoría.
- [ ] **3. Aplicar la solución defensiva correcta:** Refactorice el método en el servicio para exigir el usuario autenticado, validar que su rol sea `ROLE_DOCTOR`, comprobar que coincida con `medicoAsignado.id` y validar los datos de entrada con un DTO.

```
 // SOLUCION REFACTORIZADA Y SEGURA
@Transactional
public void actualizarNotasDefensivo(
        Long expedienteId,
        String nuevasNotas,
        String currentDoctorUsername) {
    ExpedienteMedico exp = expRepo
            .findByIdWithDetails(expedienteId)
            .orElseThrow(() ->
                new ResourceNotFoundException(
                    "Expediente no encontrado"));

    Usuario doc = userRepo
            .findByUsername(currentDoctorUsername)
            .orElseThrow(() ->
                new AccessDeniedSecurityException(
                    "Doctor no valido"));

    if (doc.getRol() != Rol.ROLE_DOCTOR ||
```

```
        !exp.getMedicoAsignado().getId()
            .equals(doc.getId())) {
        throw new AccessDeniedSecurityException(
            "Solo el medico asignado puede " +
            "actualizar las notas.");
    }

    if (nuevasNotas == null ||
        nuevasNotas.trim().length() < 10) {
        throw new IllegalArgumentException(
            "Notas: minimo 10 caracteres.");
    }

    exp.setNotasClinicas(nuevasNotas.trim());
    expRepo.save(exp);
}
```

Parte 3: Reto Autónomo

Usted debe extender la seguridad del sistema MediVault implementando de forma autónoma las siguientes dos capacidades defensivas avanzadas:

Requerimiento A: Rate Limiting Granular por Dirección IP del Cliente

El filtro de tasa actual utiliza un cubo global compartido. Usted debe modificar el `RateLimitingFilter` para que:

1. Extraiga la dirección IP del cliente remoto inspeccionando adecuadamente la cabecera `X-Forwarded-For` (en caso de proxies inversos) y, si no está presente, usando `request.getRemoteAddr()`.
2. Asigne y mantenga un cubo independiente para cada dirección IP utilizando un `Map<String, Bucket>` en memoria (`ConcurrentHashMap`).
3. Aplique un límite estricto de **5 intentos de login por minuto por IP**. Si una misma IP supera 5 intentos fallidos, debe ser rechazada con HTTP `429 Too Many Requests`.

Requerimiento B: Registro de Auditoría de Seguridad Inmutable (`SecurityAuditorService`)

Implemente un servicio de auditoría que guarde registros automáticos en la tabla `AUDIT_LOGS` ante los siguientes eventos:

1. **Intento de Acceso No Autorizado (IDOR):** Cada vez que se lance una `AccessDeniedSecurityException`, intercepte el evento y registre el nombre de usuario, la IP de origen, la acción `IDOR_ATTEMPT_BLOCKED`, el código `403` y el expediente que intentó vulnerar.
2. **Fallo de Autenticación:** Cada credencial inválida debe quedar registrada con la acción `LOGIN_FAILED` y código `401`.

Lista de Verificación del Reto (Checklist de Éxito)

- [ ] La clase `RateLimitingFilter` mantiene buckets independientes por dirección IP del cliente.
- [ ] Al probar desde una misma IP 6 solicitudes consecutivas al endpoint `/api/v1/auth/login`, la sexta solicitud recibe HTTP 429.
- [ ] La tabla `AUDIT_LOGS` registra de forma automática e inmutable los incidentes de IDOR y fallos de login con su respectiva IP y timestamp.
- [ ] Las consultas de auditoría se realizan de forma desacoplada y no degradan el rendimiento de la aplicación ni exponen datos clínicos.

Criterios de Evaluación y Rúbrica Formativa

Criterio Evaluado	Evidencia Requerida	Ponderación
Mitigación IDOR / BOLA	Implementación de `validarPropiedadRecurso` en `ExpedienteService`. Bloqueo efectivo de acceso entre pacientes con código HTTP 403 RFC 7807.	**30%**
Cabeceras HTTP y Spring Security 6	Configuración de `SecurityFilterChain` sin estado, cabeceras CSP Nivel 3, HSTS, `nosniff`, `FrameOptions: DENY` y filtro JWT.	**25%**
Validación DTO & RFC 7807	Bean Validation en contratos RESTful, captura centralizada sin fugas de stack traces ni nombres de tablas en errores 400/500.	**20%**
Rate Limiting con Bucket4j	Filtro funcional con algoritmo Token Bucket, respuesta HTTP 429 y cabecera `Retry-After`.	**15%**
Reto Autónomo (IP Bucket & Auditoría)	Control de tasa individualizado por IP y persistencia de eventos de seguridad en tabla `AUDIT_LOGS`.	**10%**
Total	**Cumplimiento de Buenas Prácticas de Seguridad Web**	**100%**